

Fable 5 Differential Review

What Fable 5 caught after Opus 5 — NphiesCore, branch feat/map-hierarchy

Prepared for Aziz · Nphies PMO · 12 August 2026

How this review was run

Every line reviewed here was written and reviewed earlier the same day by Claude Opus 5 agent fleets, behind green gates (type-check clean, 0 lint errors, ~4,400 passing tests, successful builds). Nine Fable 5 reviewers then re-reviewed all eleven session commits, one reviewer per unit of work, each given the full list of issues the Opus fleets had already documented — re-reporting a known issue counted for nothing. Every candidate finding was then attacked by two independent Fable 5 skeptics briefed to refute it and to default to refuted when uncertain; a majority refutation killed the finding. 66 agents in total.

Run statistics. 5 units reviewed (map-branch-edit, org-sidebar, theme-purple, jira-function, jira-mapper-screen). 12 raw surviving findings deduplicated to 10 — two merges, each collapsing one root cause reported from two files: (a) the stale contrast certification in mindtree.css + drag-layer.css, (b) the dead Jira mapper + the missing invisible-mark stripping in the shipped resolver. Verdicts: 9 CONFIRMED (each independently verified by two adversarial skeptic passes, most with executable reproductions), 1 CONTESTED (retained, marked, defense included). Severity after merge: 1 high, 6 medium, 3 low. Classification: 6 NEW-DEFECT, 3 NEW-RISK (real but requiring an unlikely trigger — an operator slip, a >600-field site, a non-Latin-1 email), 1 HARDENING. Every finding was checked against the repo's KNOWN-issues list by both skeptics; none duplicates a documented item. Unit test suites re-run during review: 158/158 (map-branch-edit), 28/28 (org-sidebar), 15/15 (theme-purple contrast), 90/90 (jira-function) — all green, which is itself part of the story: every finding here shipped through green gates.

Executive summary

A differential review of the nphiescore workspace — Fable 5 auditing five units of work produced by Claude Opus, with every candidate finding then attacked by two independent skeptic verification passes — surfaced ten distinct issues that survived: nine CONFIRMED, one CONTESTED. The headline finding is structural, not cosmetic: the Jira mapping screen ships a hand-rolled issue resolver while the stronger, fully-tested mapper (~1,900 lines including its 811-line, all-green test suite) has zero production callers — so every guarantee those green tests pin is absent from the screen that actually runs, including the invisible-bidi-mark stripping that makes Arabic organization names match; a Jira field that renders pixel-identical to the workspace name reports "unknown organization" in the shipped screen today. Around it: the Jira edge function echoes a misfiled API token back to every one of the 18 structure.edit holders despite its own written "never in an error body" guarantee; its pagination silently and permanently drops issues when a short page crosses the 200-issue cap; on the live map, the archive dialog and its screen-reader announcement state the opposite of what the map actually does for any branch still holding open work; a phantom "panel open" flag swallows the Escape key and speaks a false sentence to screen-reader users from the default workspace state; and the accessibility contrast certification quotes figures computed for an accent color two generations old, concealing a genuine sub-3:1 focus-indicator floor it explicitly certifies as passing. The common thread is precise: nearly every finding lives in the seam between two files or two units — dialog copy written against one module's rule while the canvas draws another's, a field populated on one side of a contract and never read on the other, a certification re-run that only refreshed the numbers it recognized. Each side is locally correct; no single test owned the interaction. One honest sentence the reader should keep: a second Opus pass, given the same differential brief and fresh eyes, would also have caught some of these — fresh context and an adversarial brief are part of the effect, and this report should not be read as a pure model-capability delta.

The findings at a glance

#	Severity	Class	Verdict	Finding	Where
1	HIGH	DEFECT	CONFIRMED	Shipped Jira mapper is a weaker parallel of a dead, fully-tested module — and it fails on real Arabic input	src/lib/jira/map.ts:264
2	MEDIUM	RISK	CONFIRMED	jira-read echoes the raw JIRA_BASE_URL in an error body — the modeled operator slip hands the Atlassian token to all 18 structure.edit holders	supabase/functions/jira-read/index.ts:425
3	MEDIUM	DEFECT	CONFIRMED	Archive dialog and its screen-reader announcement state the opposite of what the map does for any branch holding open work	src/components/mindtree/No deMenu.tsx:1020
4	MEDIUM	DEFECT	CONFIRMED	Phantom 'panel open' state swallows Escape and produces a false screen-reader announcement — reachable from the default workspace	src/pages/map/useMapLens.ts:243
5	MEDIUM	DEFECT	CONFIRMED	jira-read pagination permanently and silently drops issues when the 200-issue cap crosses mid-page	supabase/functions/jira-read/index.ts:1119
6	MEDIUM	DEFECT	CONFIRMED	Contrast certification quotes figures from an accent two generations old — including a 'non-text floor' documented as 3.29 that actually sits at 2.61, under WCAG's 3:1	src/pages/min dtree.css:1132
7	MEDIUM	DEFECT	CONTESTED	CONTESTED — Org sidebar stays pinned to a previously picked organization across every dive gesture	src/pages/map/useMapLens.ts:183
8	LOW	RISK	CONFIRMED	opFields truncates to the 600-field cap BEFORE sorting custom-first, so large sites can silently lose exactly the custom fields the operation exists to surface	supabase/functions/jira-read/index.ts:1059
9	LOW	RISK	CONFIRMED	A non-Latin-1 JIRA_EMAIL passes validation, then btoa turns every operation — including ping — into a generic 500 instead of the promised named diagnosis	supabase/functions/jira-read/index.ts:553
10	LOW	HARDENING	CONFIRMED	MindMenuRun.usage is populated, documented as what makes the archive 'audible', and read by nothing	src/components/mindtree/No deMenu.tsx:181

Classification: NEW-DEFECT = wrong today on reachable input. NEW-RISK = real but needs an unlikely trigger (an operator slip, a very large Jira site, a non-Latin email). HARDENING = correct behaviour resting on an unenforced assumption. Verdict: CONFIRMED = survived both skeptics, most with executable reproductions; CONTESTED = one skeptic's doubt is recorded beside it.

The findings in full

#1 · HIGH · NEW-DEFECT · CONFIRMED

Shipped Jira mapper is a weaker parallel of a dead, fully-tested module — and it fails on real Arabic input

`src/lib/jira/map.ts:264` (unit: `jira-mapper-screen`)

Two implementations of the same brief exist. The pure mapper in `src/lib/jira/map.ts` + `types.ts` (~1,060 lines) plus its 811-line test suite have ZERO production callers — verified by `grep` (`map.ts` even contains two NUL bytes that make plain BSD `grep` treat it as binary, which is likely why earlier reviewers found nothing). `Settings > Jira` (`JiraAdmin.tsx:80-96, 265, 336`) instead runs an independently written resolver in `src/api/jira.ts`, and the edge function keeps a third hand-rolled copy (`supabase/functions/jira-read/index.ts:927`) despite `types.ts`'s header claiming it exists precisely so the Edge Function can import it "rather than keeping a hand-drifted copy." All three copies landed in one commit (`f22e368`) with no documentation of the split. Everything the green suite pins is absent from the shipped path: no multivalued-field detection, Arabic folding hardwired on instead of the tested opt-in flag, no duplicate-pair detection (two Jira issues claiming one Organization x capability pair both read "matched"), `statusMapConflicts` never called, only the first blocking reason reported, status field id hardcoded, archived/hidden marking never reaches a row. The sharpest proven consequence: the production `normalizeName` (`src/api/jira.ts:723-733`) does NOT strip invisible bidi marks (RLM/ALM/LRM, ZWSP/ZWJ/ZWNJ, BOM) — that stripping exists and is tested only in the dead module. Node repro confirmed: an RLM-wrapped Arabic org name (`'[Arabic text] [Arabic text] [Arabic text]'` — marks Jira "routinely" adds, per the dead module's own comment) normalizes to a different key than the workspace node and the shipped screen reports `orgUnknown` for a value that renders pixel-identical. This is the vacuous-green failure mode this repo has already shipped twice with CSS. Skeptic caveat: `api/jira.ts`'s one-reason-per-issue and always-on folding are documented as deliberate there, and `JiraAdmin` has its own 936-line test file — the screen is not untested, just materially weaker than the dead contract, and the RLM gap is a hard functional defect for this Arabic-first user base.

Suggested fix. Either wire `JiraAdmin`/the api layer to `mapJiraIssues` (the stronger, tested contract) or delete `src/lib/jira/*` and port its pinned behaviors — invisible-mark stripping first (add the `INVISIBLE`-regex strip from `types.ts:369` to `api/jira.ts` `normalizeName` before whitespace collapse), then multivalued reasons, duplicate-pair detection, opt-in folding — into `api/jira.ts` where the tests can bind to the shipped path.

Why Opus missed it. Two workers built the same brief in two files; no unit owned the cross-file question of which mapper the screen imports. Both gates (`tsc`, `vitest`) stayed green because the dead half carried the tests — including a passing RLM test that proved the property against code the screen never runs. NUL bytes in `map.ts` also defeated naive `grep`.

Disposition. Resolved by the revamp's Jira decision (wave 7): the stronger, fully-tested mapper becomes the shipped path; the weaker parallel resolver is deleted; the invisible-mark stripping ships on the path that actually runs.

#2 · MEDIUM · NEW-RISK · CONFIRMED

jira-read echoes the raw JIRA_BASE_URL in an error body — the modeled operator slip hands the Atlassian token to all 18 structure.edit holders

`supabase/functions/jira-read/index.ts:425` (unit: `jira-function`)

`parseBaseUrl`'s not-a-URL branch interpolates the full raw value (`'got "${trimmed}"'`), `readCredential` (line 522) wraps it unscrubbed as `bad_base_url`, and `handle()` (lines 1250-1253) returns it verbatim to ANY caller holding `structure.edit` or `workspace.admin` — a population that gates `map_nodes` writes, not one person. The file's own header (§1) explicitly models the slip that triggers this: the owner pasting the ATATT... API token into the JIRA_BASE_URL secret box. A token has no scheme, so `new URL()` throws (reproduced), and the token lands in the error body — directly contradicting the header's §6 and README §7: "The token is never printed. Not in a log line, not in an error body." `scrub()` never runs on this path and could not help anyway (the token is not the value of `JIRA_API_TOKEN` in this scenario). The sibling `username/password` and `bad_email` branches already chose NOT to echo values, making this branch an inconsistency, not a documented trade-off. The auth-before-secrets ordering was reasoned about only for unauthenticated callers; the residual echo was waved through on the assumption the

authorized caller "is Aziz at the dashboard" — an assumption the gate does not enforce. Requires an operator slip to fire, but the exposed asset is whole-account Atlassian read access.

Suggested fix. Never echo the rejected value. Describe the shape only — "JIRA_BASE_URL is not a URL. Expected https://your-site.atlassian.net" — matching the no-echo choice the username/password branch already made.

Why Opus missed it. The header's threat analysis concluded the authorized caller is one person ("Aziz at the dashboard") — a wrong assumption that structure.edit is one person rather than the whole population that gates map writes.

Disposition. jira-read hardening unit, wave 7: the refusal describes the SHAPE of the bad value and never echoes it — matching the choice the email/password branches already made.

#3 • MEDIUM • NEW-DEFECT • CONFIRMED

Archive dialog and its screen-reader announcement state the opposite of what the map does for any branch holding open work

src/components/mindtree/NodeMenu.tsx:1020 (unit: map-branch-edit)

The confirm dialog says "It leaves the map, and so does everything beneath it" and "{n} items filed on it stop appearing on the map"; after confirming, useMapWrites.ts:141 announces via aria-live "Archived {label}. It and everything beneath it have left the map" (en + ar). But the canvas consumes the UNFILTERED node list (useMapNodes → configStore.mapNodes, config.ts:388-390), and model.ts:1204 deliberately keeps an archived entity drawn whenever it holds open work or children — a rule pinned by the model's own tests ("keeps an archived Org that still holds work, marked retired"). So an admin archiving an Organization with 12 open items watches the branch stay on the canvas, Archived-badged, all 12 items visible — while a screen-reader user has just been told it left. The guard inversion is exact: the count sentences only render when the branch holds work, which is precisely the case in which they are false; when they would be true (empty branch, which the model does drop) they are guard-suppressed. Root cause is a genuine intent conflict between modules: NodeMenu's copy was written against store/config.ts's deriveMap rule (which does drop archived children — but feeds mapChildren/mapRoots, which the canvas does not use for entities), and the 0023 DB cascade makes it worse by archiving every descendant, each populated one also staying drawn. Both skeptic passes confirmed every link of the chain.

Suggested fix. Reword confirmArchiveBody / confirmArchivedItems / branchArchived (en + ar) to state the real rule — the branch is marked archived, stops taking new work, and disappears only once nothing open is filed beneath it — or change the map model so an archived subtree is actually dropped. Add a test pinning the dialog's claims against what buildMindtree emits for an archived populated entity.

Why Opus missed it. The dialog copy was written against store/config.ts's filtering rule while the canvas draws model.ts's tree, whose deliberate opposite rule for populated archived entities lives in a different unit — a cross-file interaction no single unit owned, visible only when the branch is populated.

Disposition. Seam-fix unit, wave 8: the dialog copy and announcement are rewritten against the model's actual rule — a branch holding open work stays drawn, marked retired.

#4 • MEDIUM • NEW-DEFECT • CONFIRMED

Phantom 'panel open' state swallows Escape and produces a false screen-reader announcement — reachable from the default workspace

src/pages/map/useMapLens.ts:243 (unit: org-sidebar)

applyLens calls setMindPanelOpen(true) unconditionally, even when the lens resolves to a 'none' subject — so the store says the panel is open while Mindtree renders no panel (subjectForLens('shape', null) = {kind:'none'} → panelFor returns null, Mindtree.tsx:804). Escape's third rung (Mindtree.tsx:614) tests only lens.panelOpen, not the panel visibility Mindtree itself computes at line 911: the press is consumed (preventDefault, useMapKeyboard.ts:644-647), dive.surface() is skipped, and the live region announces "The panel is hidden." about a panel that was never showing. Concretely: a keyboard user who has dived with the camera must press

Escape twice to surface, and a screen-reader user hears a false sentence. Skeptics confirmed it is broader than filed: `DEFAULT_PREFS.panelOpen` is true, so a fresh default workspace exhibits the swallowed Escape with no chip tap at all. Decisive evidence this is an oversight, not design: `useMapUrl.ts:389` guards the equivalent inbound write with `mapLensOpensPanel` (subject `!== 'none'`), with a comment explaining exactly this hazard — the author knew it and guarded only one of the two writers. The ladder test stubs `closePanel`, so the real guard is untested.

Suggested fix. In `applyLens`, only `setMindPanelOpen(true)` when `panelSubjectFor(next, pick, focusNodeId).kind !== 'none'`; and harden the Escape rung to the visibility Mindtree already computes: `panel !== null && lens.panelOpen`. (Root cause is arguably the rung testing `panelOpen` instead of `panelVisible` — fix both.)

Why Opus missed it. Cross-file interaction no unit owned: the hook's setters cannot be observed under `renderToStaticMarkup`, and nothing ever rendered the `panelOpen=true / subject='none'` combination, so the phantom flag was invisible to every test while each file looked locally correct.

Disposition. Seam-fix unit, wave 8: the phantom panel-open state is removed; Escape and the announcement follow the real panel.

#5 · MEDIUM · NEW-DEFECT · CONFIRMED

jira-read pagination permanently and silently drops issues when the 200-issue cap crosses mid-page

`supabase/functions/jira-read/index.ts:1119` (unit: jira-function)

When `opSearch`'s 200-issue cap is crossed mid-page, the inner loop drops the unpushed tail of the crossing page — but `nextPageToken` is advanced to the page AFTER it, so a resume never sees those issues. Reproduced by running the exact loop logic against mocked pages of 100/70/100: call 1 returns 200 issues with a token pointing past page 3, and page 3's 70 dropped issues are returned by NO call, with `truncated:true` giving no hint of a gap. The precondition is real: Jira's `/search/jql` treats `maxResults` as a ceiling and shortens pages via permission filtering (acknowledged in the file's own comment near line 270) — and short pages are not even required: for `maxResults=70` (200 not a multiple of 70), the cap ALWAYS crosses mid-page with perfectly full pages. README §8 promises the opposite: "rather than quietly showing you part of your backlog as though it were all of it." The client's Load More is not yet wired (known), but the token is a shipped API contract whose stated purpose is a correct resume — once wired, it resumes from the poisoned cursor.

Suggested fix. Remember the token that fetched the current page; if the cap truncates mid-page, return THAT token as `nextPageToken` (client dedupes by key). Alternatively, stop before fetching a page when the remaining budget is smaller than a page.

Why Opus missed it. The paging loop shipped with zero test coverage (a known gap), and a hand-trace with full, evenly-dividing pages (default 50 into 200) looks correct — the loss only appears when a page misaligns the running total with the cap.

Disposition. `jira-read` hardening unit, wave 7: the paging loop honours the cap mid-page instead of dropping the remainder of a short page.

#6 · MEDIUM · NEW-DEFECT · CONFIRMED

Contrast certification quotes figures from an accent two generations old — including a 'non-text floor' documented as 3.29 that actually sits at 2.61, under WCAG's 3:1

`src/pages/mindtree.css:1132` (unit: theme-purple)

One root cause, visible from two files. The focus-stroke contrast figures in `mindtree.css` (lines 119-121, 1130-1133: 4.07/3.29/6.72) reproduce bit-exact for `#58a6ff` — the accent TWO generations ago — not the current `#9884d6/#322067`. Re-running the file's own stated sweep (140,608 hues, 5/channel, gamma-encoded blend) gives 3.24 / 2.61 / 5.34 dark: the row marked "the non-text floor," certified at 3.29, actually bottoms at 2.61 — under WCAG 1.4.11's 3:1 — because `tracks.color` is a free hex field and the sheet's guarantee is explicitly "for all of them." The same three stale figures are quoted in `drag-layer.css:65-67` under prose saying they are "quoted rather than re-derived so the two files cannot drift" (they drifted through two accent changes together), applying to the drag 'accepting outline' — the one mark telling the user a drop will be accepted; that file's count-chip row

(11.98/14.30) matches no accent era at all (true: 9.95/12.56). Git blame confirms the narrative: adjacent rows in the same headers WERE updated this session, and global.css/BRAND-NPHIES.md certified "every derived figure in those five sheets stays true to within ~0.15" while these rows drift 0.68-4.3. Skeptic trims: the 2.61 floor needs near-white track colors (the example hexes in the original scenario pass), and a global :focus-visible outline exists as a second indicator at 5.34:1 — though the sheet itself calls that outline engine-unreliable. The false certification stands regardless; both skeptic passes reproduced every number to the digit.

Suggested fix. Re-run the sweep with #9884d6/#322067 and correct mindtree.css:119-121/1130-1133 and drag-layer.css:57-58/65-67 (3.24/9.41, 2.61/8.07, 5.34/13.65; count chip 9.95/12.56). Since the dark hover-fill floor is genuinely under 3:1, give the focus stroke and accepting outline the same escape the dive-rail thumb got (a 2px --bg-elev backing ring) or cap the hover fill share. Narrow the '<0.15 drift' claim in global.css/BRAND-NPHIES.md to rows actually re-verified.

Why Opus missed it. The recompute pass only refreshed figures matching the accent it was replacing (#7a8ad6-era numbers); rows stale since the 07-31 repaint matched nothing it was looking for and were assumed to be independent measurements. drag-layer.css compounded it: its figures are declared as quotes owned by mindtree.css, so the updater synchronized only the one line whose number matched and trusted the rest.

Disposition. Theme fix, wave 8: every certified figure recomputed wholesale against the current accent, including the real 2.61:1 focus floor; the lesson (recompute, never pattern-match) becomes standing gate policy.

#7 · MEDIUM · NEW-DEFECT · CONTESTED

CONTESTED — Org sidebar stays pinned to a previously picked organization across every dive gesture

src/pages/map/useMapLens.ts:183 (unit: org-sidebar)

The selection-drop rule ("the drill-in MOVES → dropped") is keyed to focusNodeId, but after the camera rebuild all dive gestures — Enter/click on a department, the dive rail, the canvas breadcrumb — are pure camera flies that never write setMindFocus. Mechanics verified by both skeptics: tap Organization A, dive into department B two worlds away, and the screen is framed inside B with A's account manager, vendor, and use-case matrix still beside it — which the file's own header calls "the stale-panel defect" rule 1 exists to prevent. The drop only fires from MapList focus, the panel's own breadcrumb, or a hand-edited ?focus= URL. CONTESTED because the second skeptic makes a serious design defense: dive gestures are discrete shortcuts to the same camera states wheel/pinch zoom reach continuously (a details card that closes on every zoom notch would be absurd), the camera-era spec deliberately decouples panel and camera in both directions, and the Escape ladder's ordering (dismiss card → close panel → surface) only makes sense if the panel legitimately survives dives — leaving the header's "reader dove somewhere else" prose as pre-camera vocabulary drift rather than a live defect. Skeptic's doubt in one line: the surviving card is clearly titled with the picked node's own name and answers "what is the state of this one thing," which a camera move does not change — so this may be documented-intent drift, not broken behavior.

Suggested fix. If treated as a defect: key the drop on the FRAMED world (thread worldAt-derived framedId into useMapLens), or have dive.into / rail / crumb clear the selection when the target is outside the selected node's ancestor chain. If treated as design: rewrite the rule-1 header prose so it stops describing pre-camera behavior the code no longer has.

Why Opus missed it. Wrong assumption carried across the camera rebuild: rule 1 was written when a dive wrote setMindFocus; no test drives useMapLens together with the camera verbs, where 'dove somewhere else' no longer moves focusNodeId.

Disposition. Resolved by design in wave 4: the sidebar follows the dive via the lens-preserving subject change, which settles the contested behaviour in the direction the owner expects.

#8 · LOW · NEW-RISK · CONFIRMED

opFields truncates to the 600-field cap BEFORE sorting custom-first, so large sites can silently lose exactly the custom fields the operation exists to surface

supabase/functions/jira-read/index.ts:1059 (unit: jira-function)

fields.slice(0, FIELDS_CAP) at line 1060 runs before the custom-first sort at 1065, so on a site with >600 fields the truncation falls on Jira's unspecified wire order. Reproduced: 700 fields with customs late in the array returns 600 fields with customCount 0 — every custom field dropped — while the response looks complete and custom-first. The inline comment at 1062-1064 says custom fields "are what this operation exists to surface," and README §6 makes the default call (not customOnly:true) the documented first step. Sorting before slicing preserves the contract at zero cost. Skeptic trim: not fully silent — the response carries truncated:true, returned, totalOnSite, and customCount, so a careful consumer can detect a cut, but not WHICH fields were cut. Requires an instance exceeding 600 fields, which large Jira Cloud instances do reach.

Suggested fix. Reorder: filter customOnly, then sort custom-first, then slice(0, FIELDS_CAP) and compute truncated — truncation then drops system fields last.

Why Opus missed it. The cap and the sort were written as two tidy adjacent blocks, each individually correct; the order dependency only matters on a >600-field site, which no fixture models.

Disposition. jira-read hardening unit, wave 7: sort custom-first, then truncate.

#9 · LOW · NEW-RISK · CONFIRMED

A non-Latin-1 JIRA_EMAIL passes validation, then btoa turns every operation — including ping — into a generic 500 instead of the promised named diagnosis

supabase/functions/jira-read/index.ts:553 (unit: jira-function)

looksLikeEmail's loose regex accepts Unicode localparts (verified: an Arabic-script address passes — relevant, since the app ships an Arabic locale for exactly this operator). basicAuthHeader then calls btoa, which throws InvalidCharacterError on any character above U+00FF. The throw escapes: jiraCall builds the secrets array at line 826, four lines OUTSIDE its try block (opens at 830), so the error propagates to handle()'s catch-all (line 1288) and every op answers 500 "Something went wrong reading from Jira" — a misdiagnosis (Jira was never contacted) that breaks the file's own contract at line 151: "A MISSING SECRET IS A DIAGNOSIS, NOT A 500." The bad_email branch exists precisely to name a malformed JIRA_EMAIL; this input evades it. Reproduced live in Node; Deno's btoa has identical semantics. Skeptic trim: Atlassian account emails are ASCII in practice, so this needs an operator paste slip — but that is exactly the case the readCredential machinery promises to diagnose by name.

Suggested fix. Either restrict looksLikeEmail to Latin-1/ASCII with a bad_email message, or build the Basic header via TextEncoder + base64 of UTF-8 bytes (which is also what Jira actually accepts for such accounts).

Why Opus missed it. looksLikeEmail was deliberately loose ('a typo check, not RFC 5322') and no one connected its Unicode acceptance to btoa's Latin-1 ceiling two functions away — a cross-function interaction no single unit test owned.

Disposition. jira-read hardening unit, wave 7: a non-Latin-1 email is refused by name at validation instead of dying as a generic 500 in the encoder.

#10 · LOW · HARDENING · CONFIRMED

MindMenuRun.usage is populated, documented as what makes the archive 'audible', and read by nothing

src/components/mindtree/NodeMenu.tsx:181 (unit: map-branch-edit)

NodeMenu fills usage on the confirmed archiveBranch run (line 788), and the field's docstring promises "the live region repeating them is what makes the act audible to a reader who cannot see the branch leave." The only consumer, useMapWrites.runMenu (useMapWrites.ts:123-143), never touches run.usage — it announces only the label, no counts — and no test anywhere references the field. So a blind reader confirms a dialog naming 3 branches and 12 items, then hears an announcement naming neither. Both ends document a contract neither fulfills: useMapWrites' own comment says the run "hands the answer over... so this function neither asks again nor counts again," then discards the handed-over counts. Compounds finding 3: the count-free announcement is also the one making the false "have left the map" claim. Dead cargo plus doc-promise drift; no crash.

Suggested fix. Either have useMapWrites' archiveBranch arm interpolate run.usage's counts into the archived announcement (with the same count>0 guards archiveConfirmCopy uses), or delete the usage field and its

contract prose from MindMenuRun. Pairs naturally with the finding-3 rewording.

Why Opus missed it. The field was added in NodeMenu's unit and the announcement belongs to useMapWrites' unit; each side's comment asserts the other side does the speaking, and no test spans the seam between the two files.

Disposition. Seam-fix unit, wave 8: the populated field is either read by the announcement it was built for, or deleted.

What was attacked and found sound

Half the value of a differential pass is the attack angles that were run to ground and came back clean — they are evidence, not filler.

WHAT WAS REVIEWED AND FOUND CLEAN — per unit. Half the value of this pass is the attack angles that were run to ground and came back sound. MAP-BRANCH-EDIT (2 findings: #3, #10). All unit files read at HEAD (actions.ts structural-verb section, actions.test.ts, NodeMenu.tsx, QuickAdd.tsx branch mode, node-menu.css, useMapWrites.ts, useMapOverlays.ts, Mindtree.tsx wiring, en/ar mindtree.json); all three test files re-run, 158/158 green. Clean: permission races (useHasPerm fails CLOSED — a not-yet-loaded Director sees no structural verbs); confirm-then-write gap (archive writes by immutable id, idempotent, cascade prose accurate against the real 0023 triggers); optimistic-vs-pessimistic paths correct on both entry-add and branch-add/archive; composer focus across dive/relayout; dismiss-before-unmount focus restore in both portals; memo-keyed context (no stale closures); Arabic plural forms complete on both cascade-count keys with correct zero-guards. Noted as pre-existing and out of scope: leaf "Mark as done" routes around a hidden/retired Done status. Test-quality: actions.test.ts genuinely adversarial; the one gap (archiveConfirmCopy untested) is the hole finding #3 shipped through. ORG-SIDEBAR (2 findings: #4, #7-contested). All unit files read at HEAD plus session diffs; suite re-run 28/28. Clean: isDiveTarget edge cases (zero-child entities, +N-more folds unconstructible as structural children, mixed-children dive is designed and pinned); the render-phase selection clear's ref-before-setter mechanics under StrictMode double-invoke (the defect found is in its INPUT, not its mechanics); Enter-vs-click funnel through one activate(); concurrent delete/archive of the focused node (no crash, honest title flip, request-token guard against stale fetches); phone half-detent precedence; vacuous-test sweep (sheet gate cannot pass on an empty glob, namespace gate real, all panel keys present in both bundles). Pre-existing, not a session catch: Mindtree's dive useMemo is vacuous (dep list identical at the base commit). THEME-PURPLE (1 merged finding: #6). Every quoted number in the unit recomputed independently, including the full 140,608-hue sweep. Clean: all 40+ NEW session ratios in global.css, BRAND-NPHIES.md, contrast.test.ts, map-altitude.css, mind-ring.css exact to the second decimal; hue-window math correct including negative-modulo and grey-guard branches, tests 15/15 green off real disk reads (not vacuous); accent-soft composites moved slightly UP everywhere (no regression); zero old-accent leftovers; TrackEditor's 8 pairs byte-equal to declared tokens, including a quietly-fixed light red; AR string present for colorPurple. Trivia noted, not filed: "12 picker swatches" (now 13); BRAND doc's use-count off. The two filed files are one root cause: a partial recompute that re-certified two-generation-old figures as green. JIRA-FUNCTION (4 findings: #2, #5, #8, #9). All 90 unit tests re-run green; index.ts, index.test.ts, README.md checked line-against-line. Clean: the allow-list validator (frozen ENDPOINTS, strict === op matching, ASCII-only field ids, no prototype-pollution path, no caller text reaches a URL path); JQL exposure is inherent and documented (gate-choice question already on the known list); has_perm runs AS the caller and fails closed on both error and null paths, secrets read only after the gate; page-count cap enforceable even against a stalled cursor; the source-text tests are NOT vacuous (empty-read fails, comment-stripping genuinely works). Noted but not filed: per-response BYTE size is unbounded (res.text() buffers; fields:["*all"] multiplies) — a provable OOM input is hard to state given Jira's own field limits, so the header's memory worry is only half-mitigated. README verified against code except the two contradictions folded into findings #2 and #5. JIRA-MAPPER-SCREEN (1 merged finding: #1). CAVEAT: this unit's coverage statement arrived truncated in the review pipeline — the finding itself was verified exhaustively by both skeptic passes (grep-verified dead code, node-repro of the RLM mismatch, feature deltas spot-checked, commit archaeology), but the unit's clean-bill list beyond the filed finding is not fully known and should not be assumed comprehensive.

Honest caveats

Four things the reader should weigh. (1) Attribution: a second Opus pass with the same differential brief would also have caught some of these findings — fresh eyes, an adversarial brief, and permission to run to ground across unit boundaries are part of the effect. The cross-file seam findings (#1, #3, #4, #10) are the strongest evidence of a genuine differential, since they required refusing each file's locally-correct story; the numeric recomputes (#6) required tooling discipline more than model capability. This report is not a controlled model comparison. (2) Severity trims the skeptics themselves recorded: the 2.61:1 contrast floor (#6) needs near-white track colors and a second (engine-unreliable) focus outline exists; opFields (#8) leaves a truncated:true tell; the token echo (#2) requires an operator slip and reaches only authenticated high-trust users; the btoa 500 (#9) requires a non-Latin-1 email, atypical for Atlassian accounts; the pagination loss (#5) needs a page boundary misaligned with the 200 cap, though maxResults=70 triggers it with perfectly full pages. (3) One finding (#7) is CONTESTED: the mechanics are verified, but a credible reading says the surviving sidebar is camera-era design and only the header prose is stale — it is ranked and worded accordingly. (4) Scope: the jira-mapper-screen coverage statement was truncated in the pipeline, so that unit's clean-bill inventory is incomplete here (its one finding is fully verified); and this synthesis trusts the two skeptic verification passes per finding — I did not independently re-execute their reproductions.

Where every finding goes

All ten findings are absorbed into the approved revamp plan (waves 4, 7 and 8), each with a named owner-wave rather than a backlog entry. #1 is resolved by the plan's Jira mapper decision: the stronger, fully-tested mapper becomes the shipped path. #2, #5, #8 and #9 form the jira-read hardening unit in wave 7. #3, #4, #7 and #10 land in the canvas/lens seam-fix unit in wave 8. #6 forces the rule that contrast certifications are recomputed wholesale, never pattern-matched. The process lesson is applied to every future build wave: gate briefs must name cross-file interactions as first-class review targets, because all ten findings lived in the seams between units — each side locally correct, no test owning the interaction.